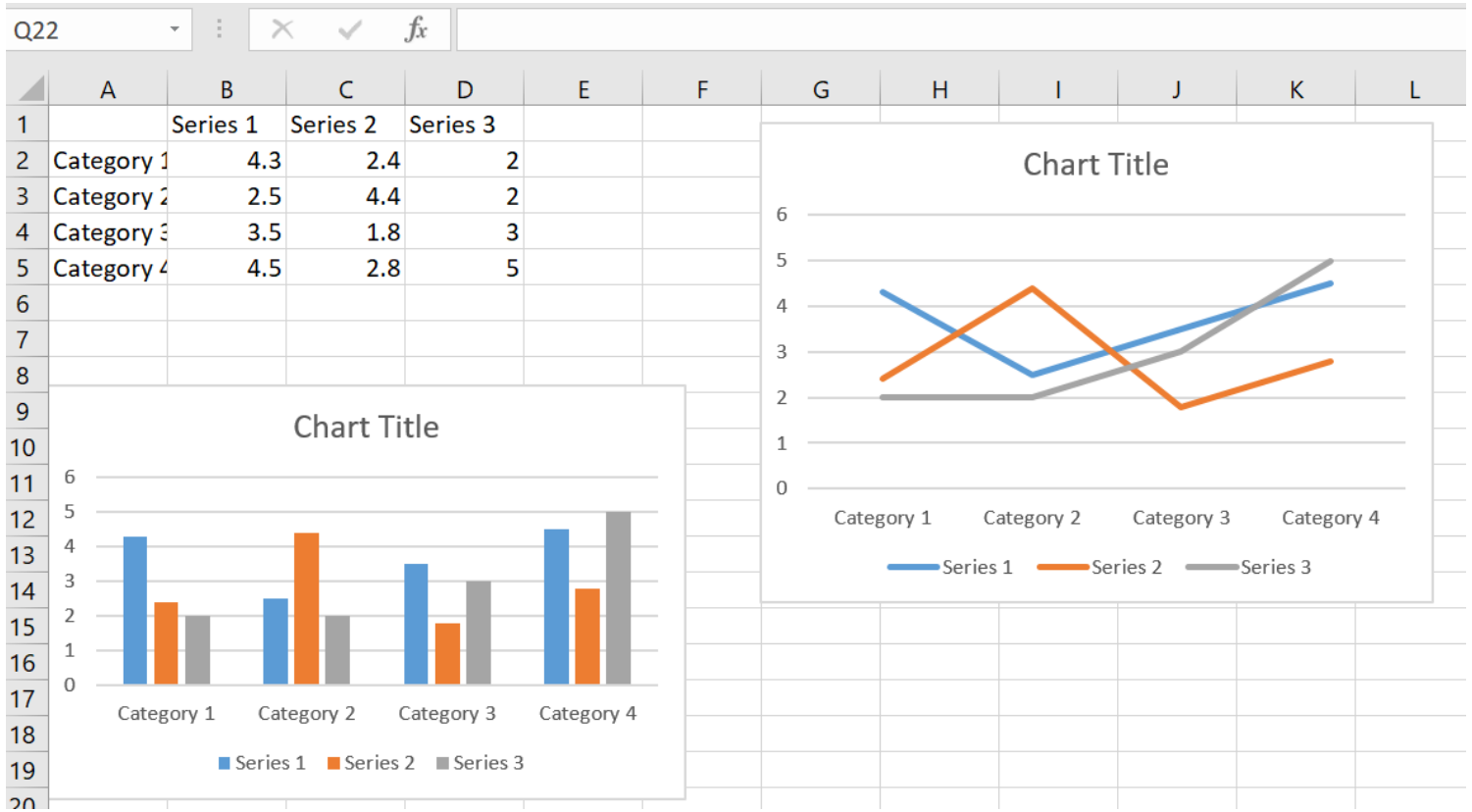
Lecture 15: MVC Design Patterns

Liu Yang

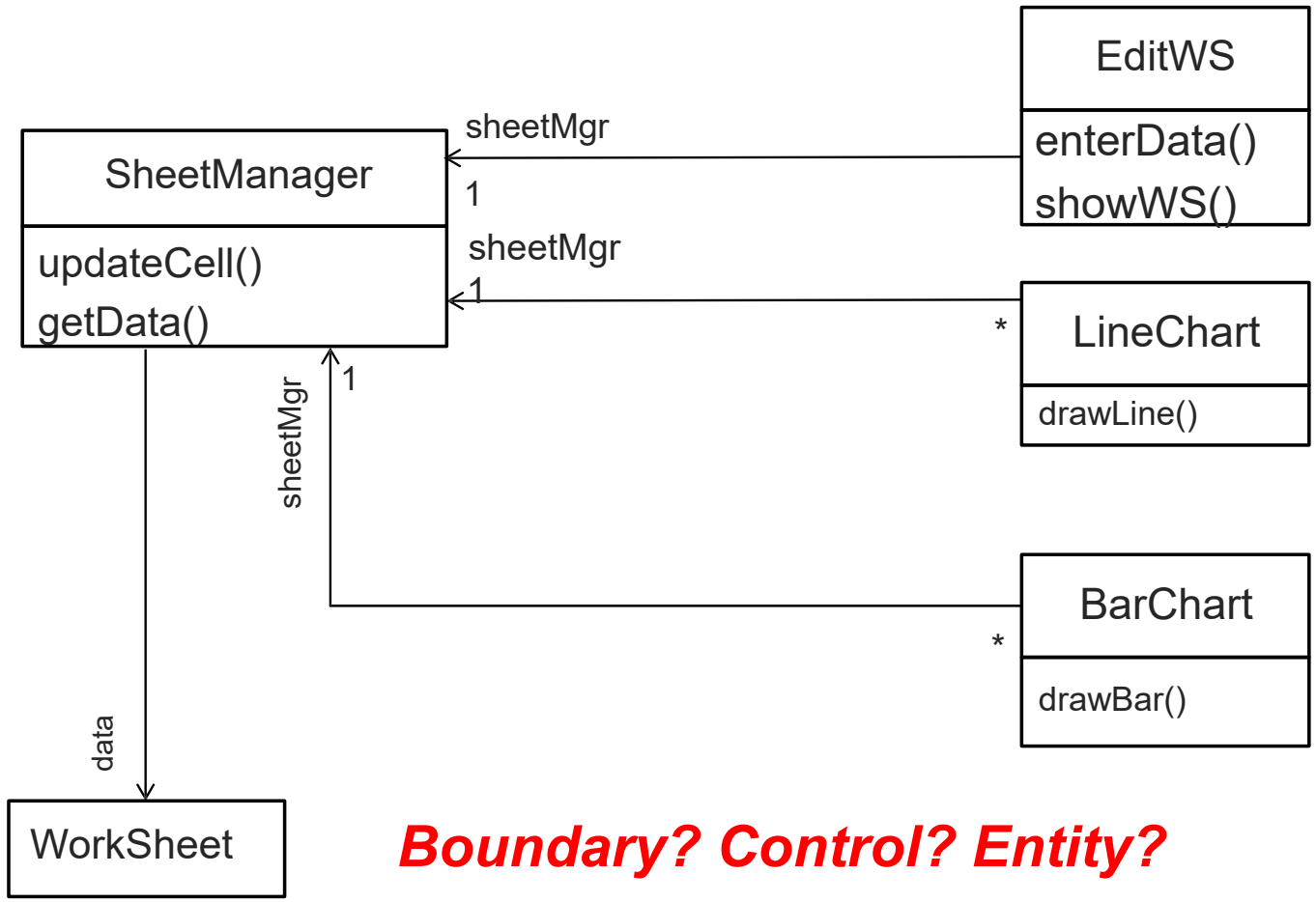
Interactive Event-Driven Systems

- UIs are prone to change
  - The same data can be presented differently
  - Changes to UI must be easy and even possible at runtime
  - The application display and behavior must reflect data changes immediately
  - The same presentation can have different look and feel
  - The application reacts to user input differently
- UI should NOT be tightly coupled with core functionality and data in order to allow for the above changes

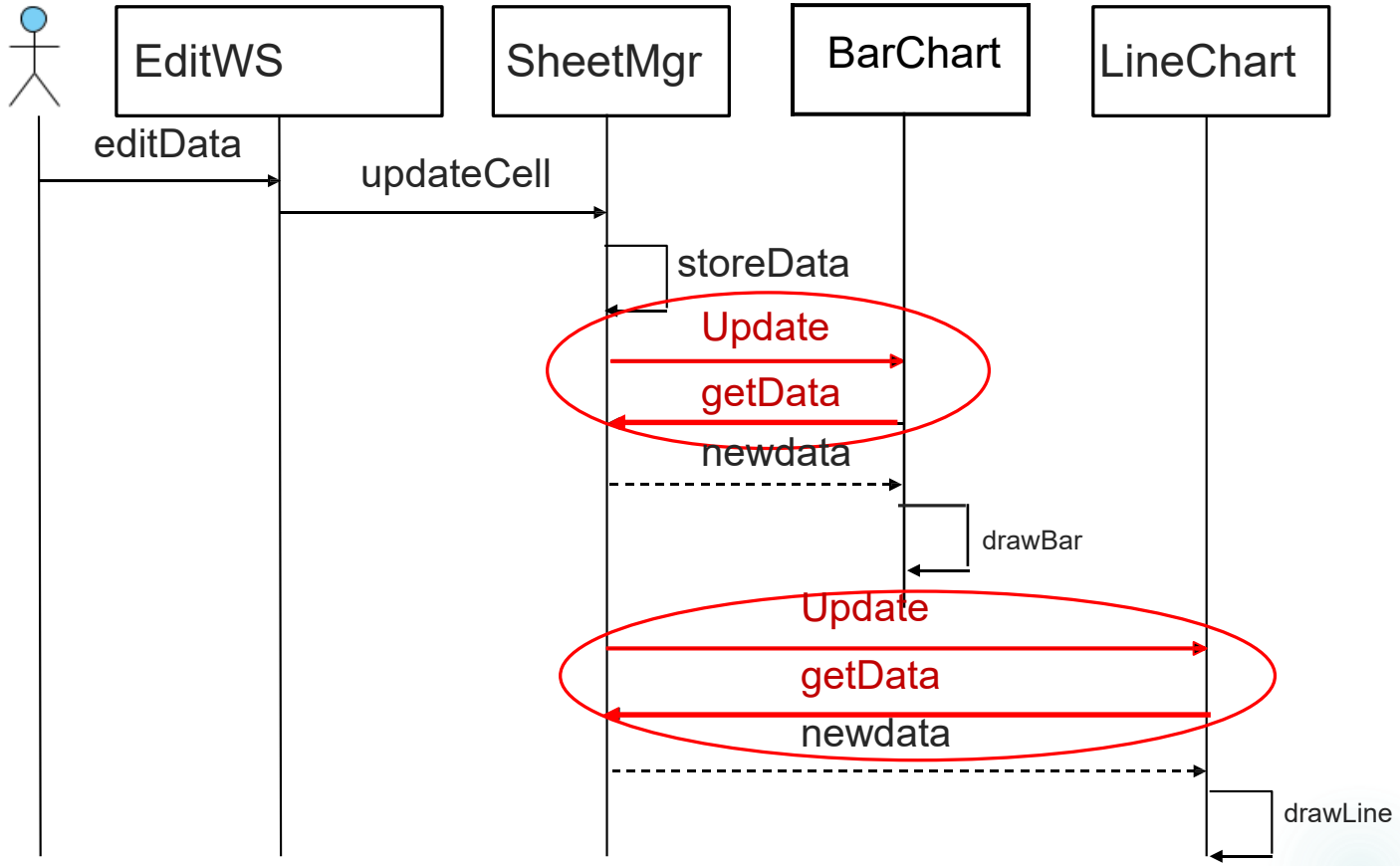


Excel Example

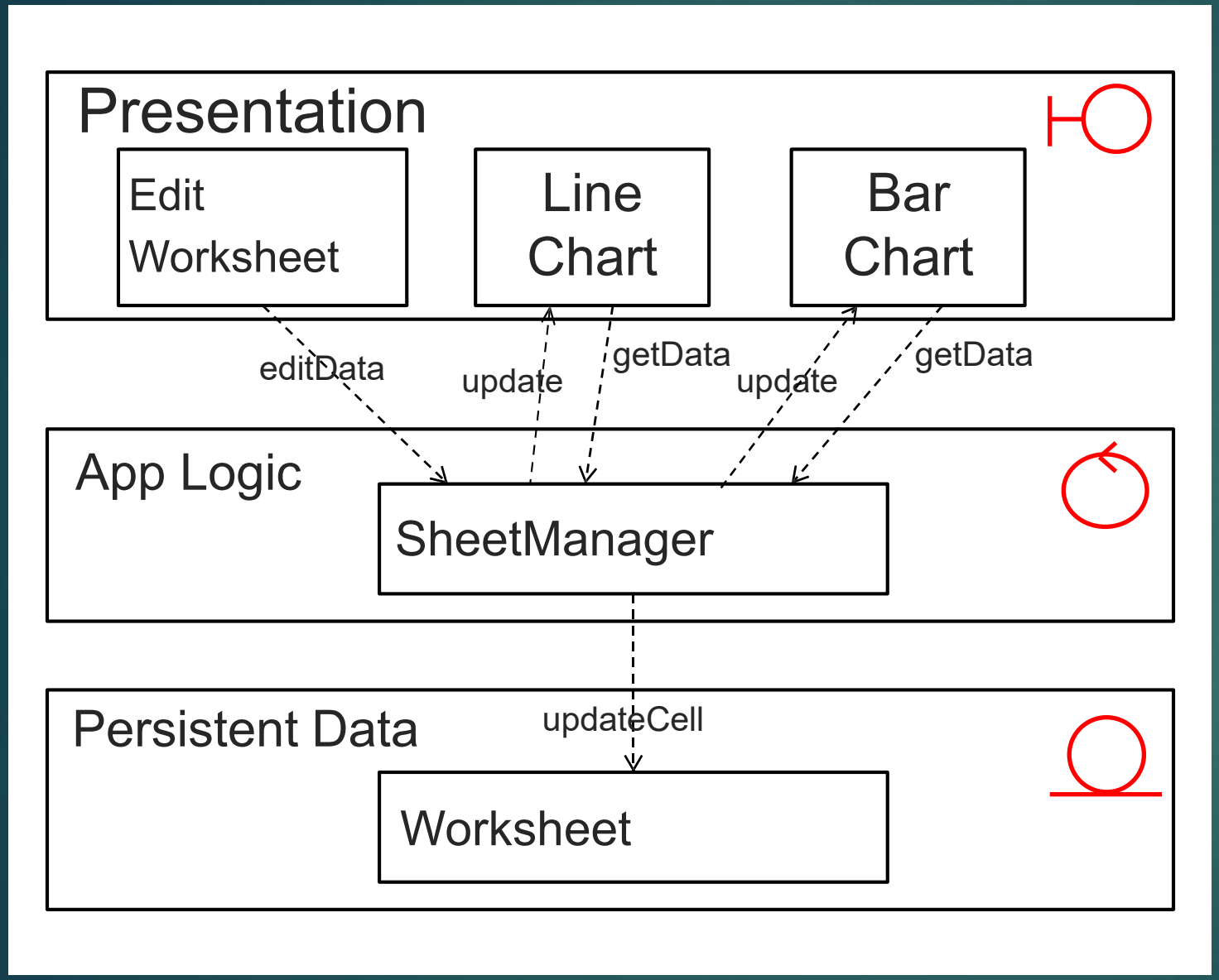
Excel Conceptual Model



Boundary? Control? Entity?

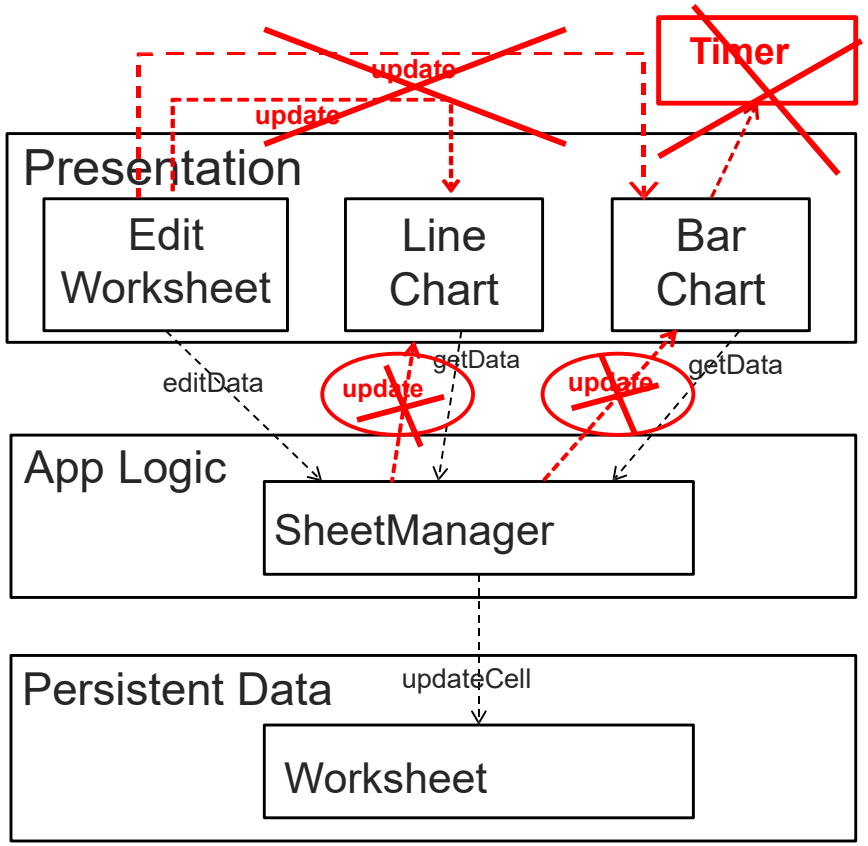


Update Worksheet – Sequence Diagram



Excel Layered Architecture

Excel Layered Architecture

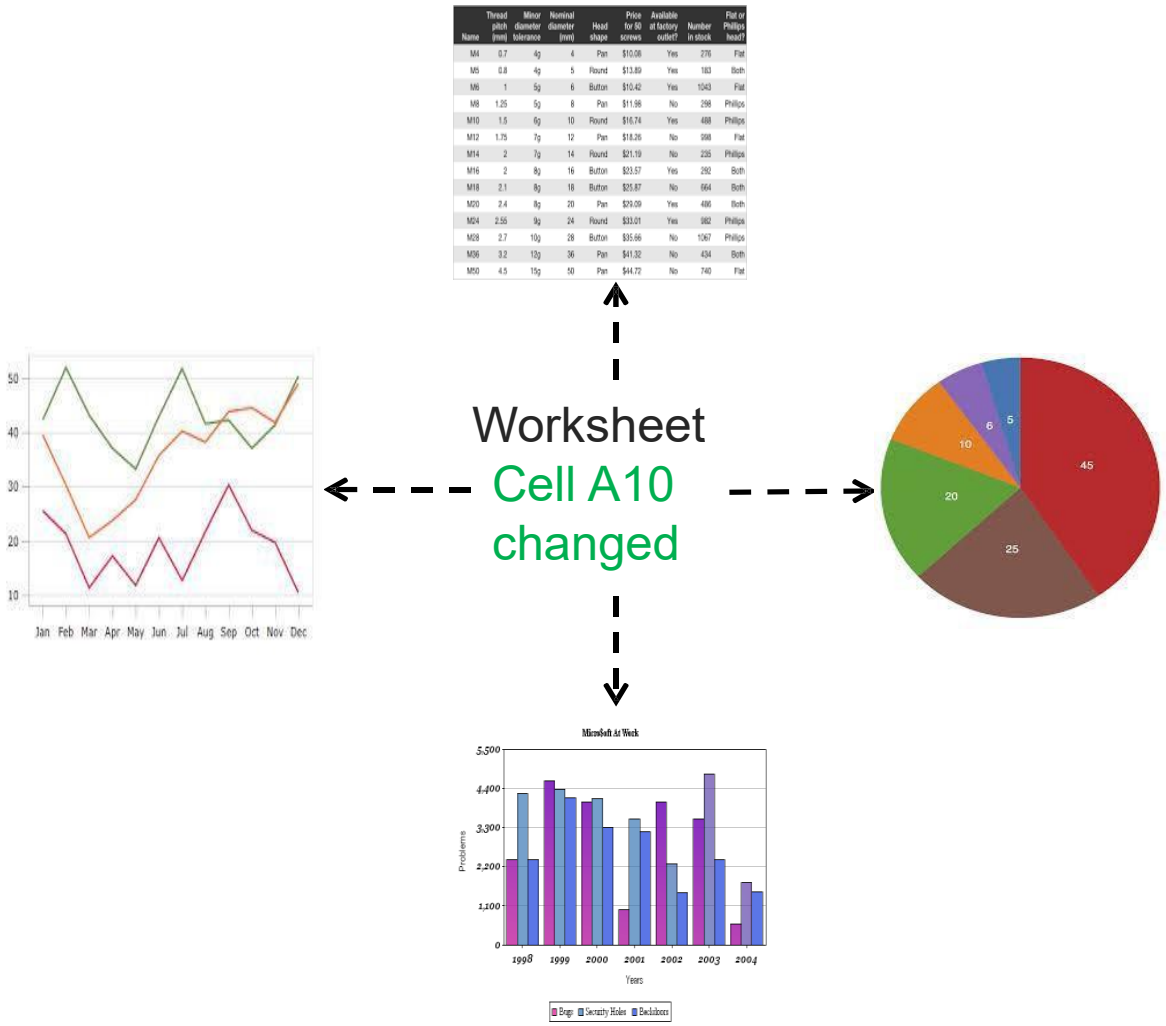


Timer: check every 10 seconds?
Slow refresh, or excessive polling.

Views calling one another?
A complex UI creates dependencies.

Presentation ↔ application logic
Direct callbacks create tight coupling.
Which pattern can help?

What do We Really Need?



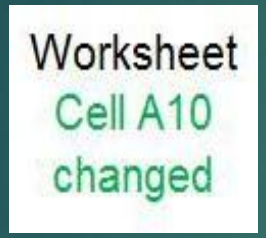
We need many views to receive an **update** when worksheet **changes**

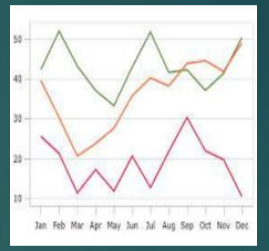
What are the Design Problems?

- Loose coupling between a **worksheet** (subject) and its dependent **views** (observers)
  - **Worksheet** wants to notify **views** the data change once it occurs, but views of a worksheet can be created/deleted freely and constantly.
  - **Views** want to show the data change immediately once it occurs, but they do not know when the data change will occur.

What design pattern can help?

Identify the Design Pattern





Subject

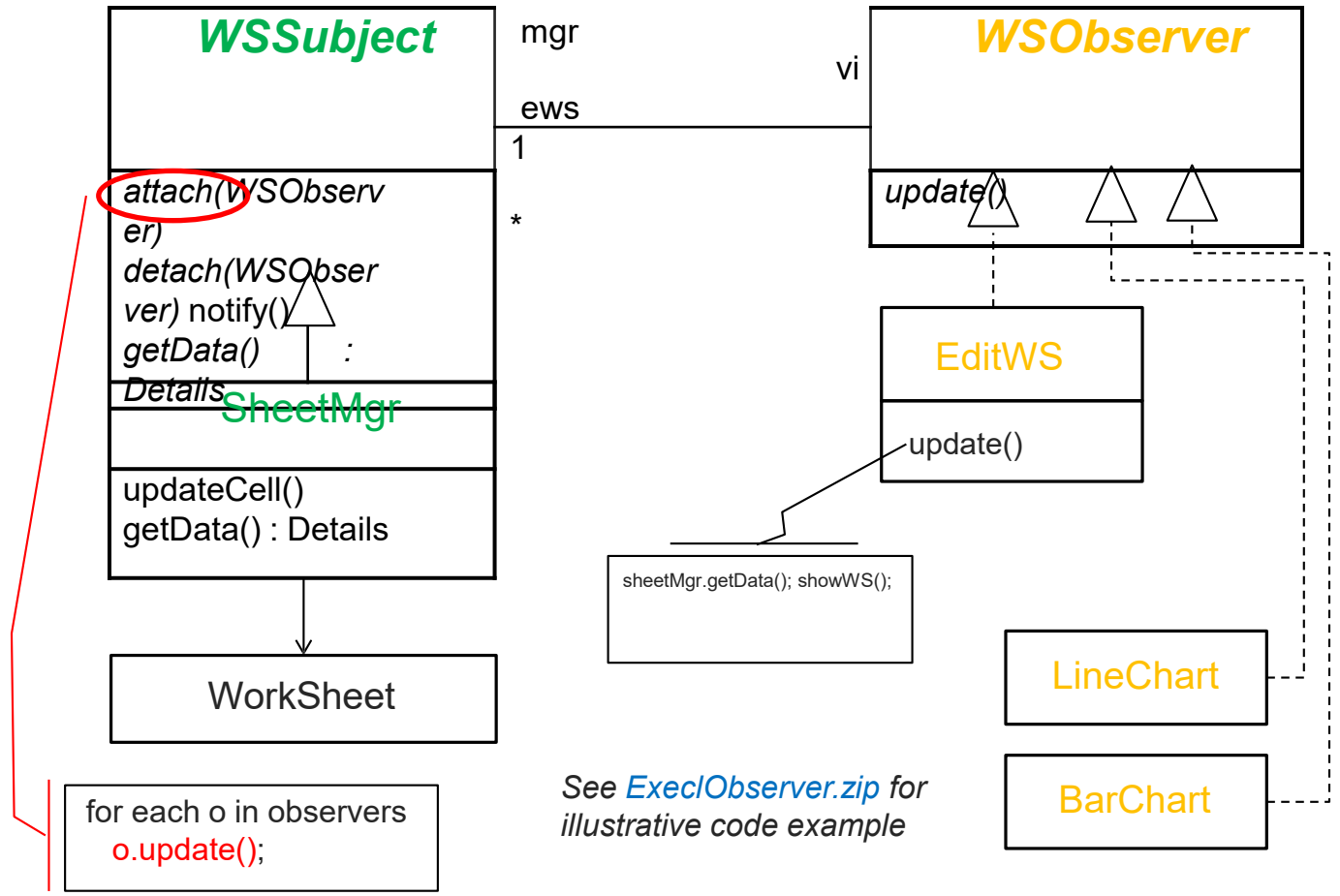
- I do not care in what format the data is visualized and how many views are visualizing the data
- I want to let whoever is visualizing the data know the latest data change when it occurs
- It is up to views to decide what to do with the data change

Observer

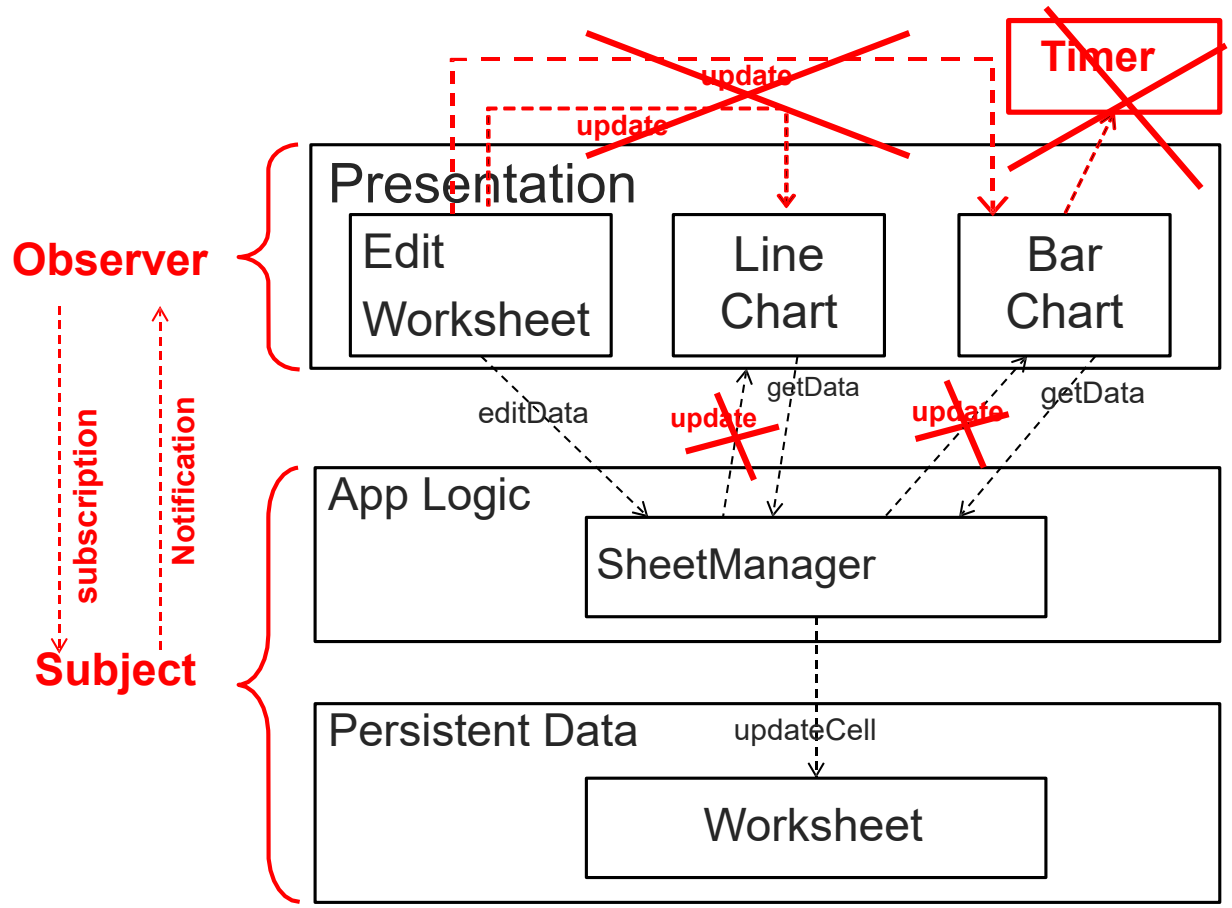
- I can be created and deleted for data visualization freely
- I want to update the chart with the latest data change, but I do not know when the data change will occur
- I cannot constantly check the data change because I do not know how frequent I should check

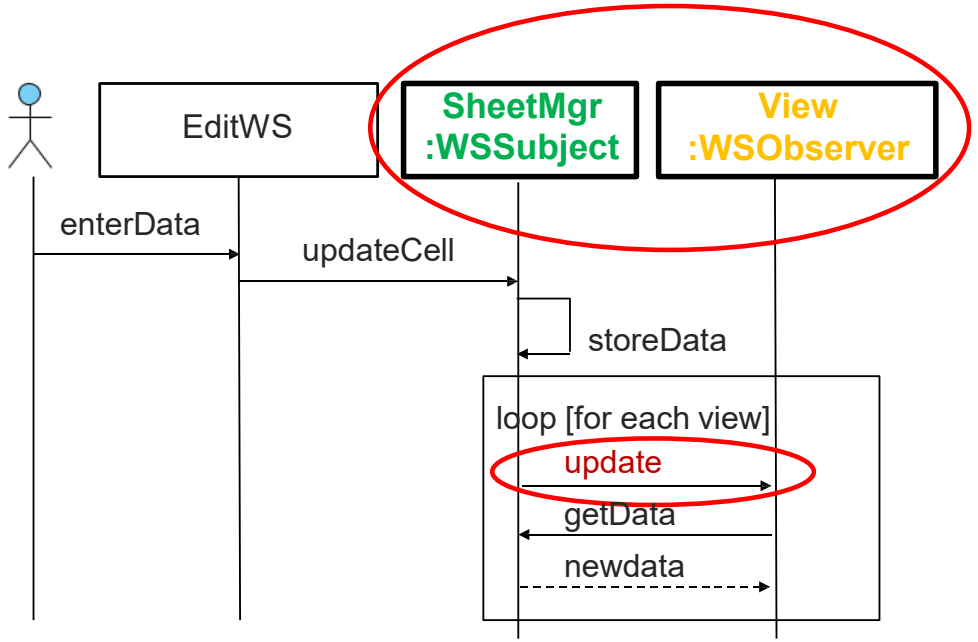
Loose coupling *is a benefit for both sides!*

Observer Pattern – Excel Worksheet – Views



Excel Architecture





*SheetManager (or other manager classes) and views communicate as **WSSubject** and **WSObserver** objects, not specific manager classes or views!*

Observer Pattern – Update Worksheet

All Problems Solved?

- UIs are prone to change
  - ✓ The same data can be presented differently
  - ✓ Changes to UI must be easy and even possible at runtime
  - ✓ The application display and behavior must reflect data changes immediately
  - ? The same presentation can have different look and feel
  - ? The application reacts to user input differently

Which pattern supports these behaviour?

All Problems Solved?

- UIs are prone to change
  - ✓ The same data can be presented differently
  - ✓ Changes to UI must be easy and even possible at runtime
  - ✓ The application display and behavior must reflect data changes immediately
  - ? The same presentation can have different look and feel
  - ? The application reacts to user input differently

Strategy Pattern (interchangeability)!

What We Have Discovered

• One problem with layering (layered architecture) is that many situations require up-calls from lower to higher layers – *dependency,* e.g. Excel worksheet – *update*

• How do we allow up-calls without creating dependencies between lower and higher layers?

• ***Apply design pattern – observer pattern (loose coupling between two layers)***

• **Observer pattern** enables loose coupling between Excel worksheet and views, but it DOES NOT solve all problems – may need more than one design pattern!
E.g. observer pattern + strategy pattern

Model–View–Controller (MVC) Architecture

- Main Goal
  - To facilitate and optimize the implementation of interactive intensive systems, particularly those that use multiple synchronized presentations of shared information.
- Key Idea
  - The separation of the Model from View and Controller components allows multiple Views of the same Model – It separates presentation and interaction from the data. If the user changes the Model via the Controller of one View, all other Views dependent on this data should reflect the changes.

Model-View-Controller (MVC) Architecture

- **Separation of UI from the core data model**
  - Need to support multiple different user interface (e.g. desktop GUI, web browser, PDA, cell phone, etc.) – Core functionality should be reusable across all different interfaces.
  - Interface details changes frequently – Changing UI details should not affect core functionality.
  - It should be easy to add a new user interface.

Model-View-Controller (MVC) architecture model describes separation of UI from core data model

Model–View–Controller (MVC) Architecture

- **MVC Components**

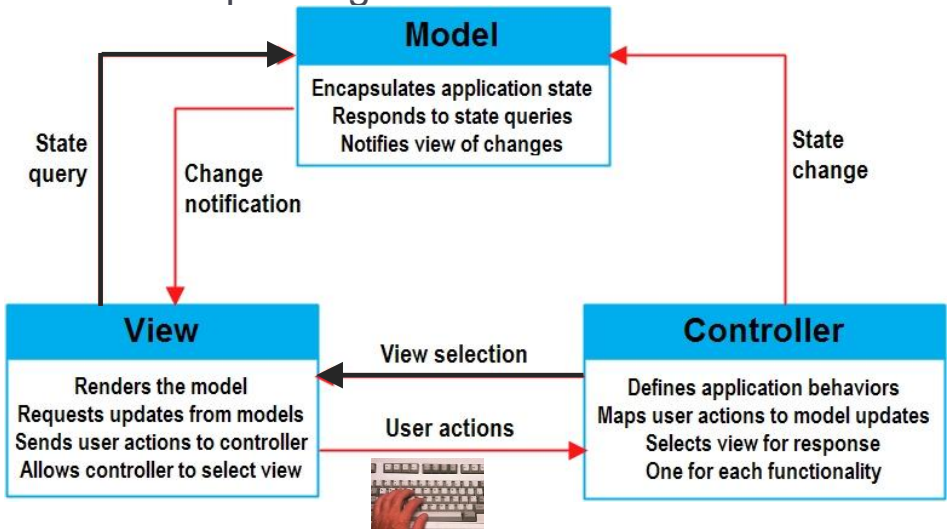
Boundary? Control? Entity?
(View, Controller) (Model)

  - Encapsulate core Functionality (App Logic) + Data in **Model**.
  - Implement UI in **View** (present data) + **Controller** (react to user input).

Model	Contains the processing (operations) and the data involved.
View	Presents the output – Defines and manages how the data is presented to the user; each **View** provides specific presentation of the same **Model**. Each **View** "*observes*" the **Model** –Whenever the data **Model** changes, all **Views** immediately **notified** and so they can update their graphical presentation.
Controller	Manages user interaction – Captures user input (events-> mouse clicks, key presses, etc.) and passes these interactions to the **View** and the **Model**. Each **View** is associated to a **Controller** that captures and processes user input and modifies the data **Model**. The user interacts with the system solely through **Controllers**.

Model-View-Controller (MVC) Architecture

- **Controllers** typically implement event-handling mechanisms that are executed when corresponding events occur.

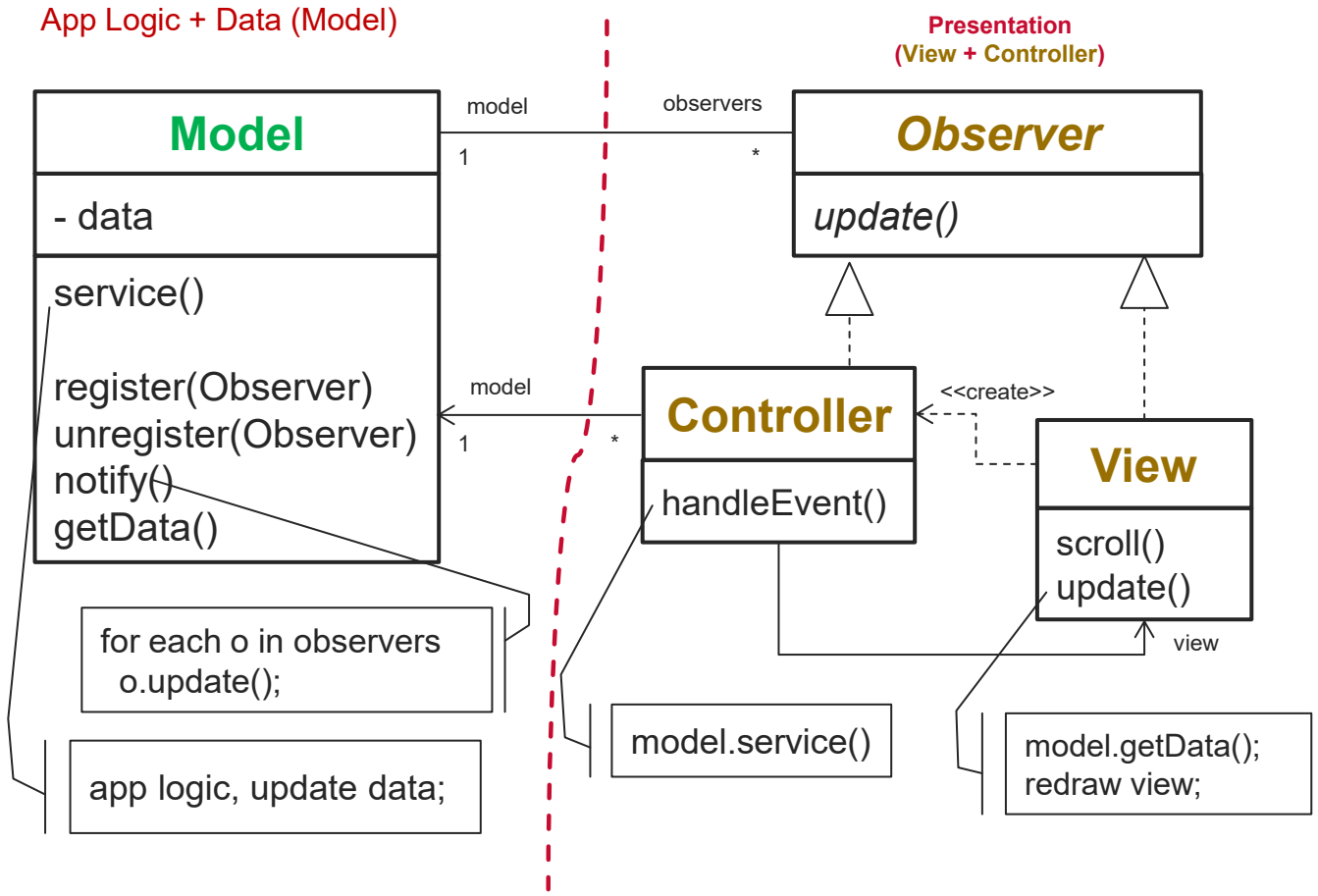


- Changes made to the **Model** by the user via **Controllers** are directly propagated to the corresponding **Views**. The change propagation mechanism can be implemented using the **observer pattern** (via subscribe/notify protocol).

Model-View-Controller (MVC) Architecture

- **MVC architecture style is non-hierarchical (triangular)**
  - **View subscribes** for changes to the **Model** (via Observer subscription mechanism)
  - **Controller** gathers input (or change) from the users and updates the **Model**.
  - **Model** updates the change and **notifies** all subscribers (the **Views**) of the change (via Observer notification mechanism)
  - **View** is notified and updates its display; the user sees the change.

MVC Architecture



Observer: Model and observers

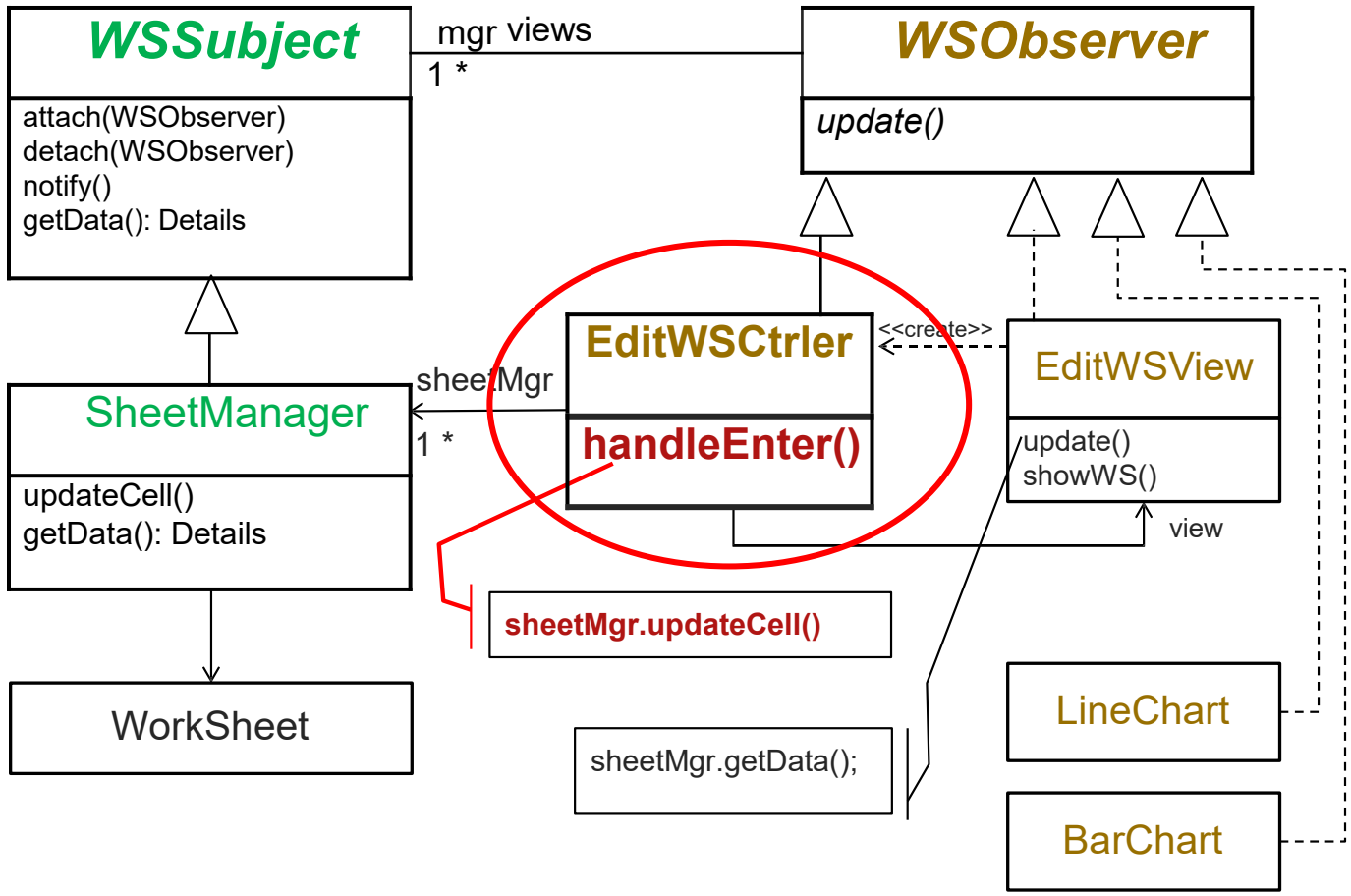
Strategy: View and Controller

Observer + Strategy Patterns in MVC

- **Observer Pattern (View–Model relationship):**
  - **Model** uses the Observer Pattern to keep the **View(s)** updated.
  - When **Model** (Subject) data is changed, it updates all the **Views** (Observers, via notification mechanism) – Allows multiple **Views** to the same **Model**.

- **Strategy Pattern (View–Controller relationship):**
  - **View** and **Controller** use the Strategy Pattern as the **View** is concerned with the visual aspects (display data) and delegates the interface behaviors to the **Controller**.
  - The **View** (Context) uses the **Controller** (Strategy Interface) to implement a specific type of response (or strategy objects). The **Controller** can be changed to allow **View** to respond differently to user input.

Excel MVC


WSSubject
attach(WSObserver)
detach(WSObserver)
notify()
getData(): Details
mgr views
1 *
WSObserver
update()
EditWSCtrler
handleEnter()
<<create>>
EditWSView
update()
showWS()
view
sheetMgr
1 *
SheetManager
updateCell()
getData(): Details
sheetMgr.updateCell()
WorkSheet
sheetMgr.getData();
LineChart
BarChart

Model-View-Controller (MVC) Architecture

- Design patterns in MVC
  - **Observer**
    - View – Model; Controller - Model
  - **Strategy**
    - View – Controller; View – Look & Feel
  - **Abstract Factory**
    - Create controller or look & feel for a view
  - **Composite**
    - Nested views (e.g. Panel contains Button; Nested Panels)
  - **Decorator**
    - Attach additional UI functionalities (e.g., scrolling) to a view
  - **Command**
    - Support undo/redo user actions

Observer, Strategy and Abstract Factory are the key patterns in MVC

Other patterns also support the design of interactive systems.

MVC

- Pros
  - Simultaneous development Multiple views for a model – Models can have multiple views
  - High cohesion
    - MVC enables logical grouping of related actions on a controller together. The views for a specific model are also grouped together.
  - Low coupling
    - The very nature of the MVC framework is such that there is low coupling among models, views or controllers
- Cons
  - Code navigability and pronounced learning curve
    - The framework navigation can be complex because it introduces new layers of abstraction and requires users to adapt to the decomposition criteria of MVC.
  - Multi-artifact consistency
    - Decomposing a feature into three artifacts causes scattering. Thus, requiring developers to maintain the consistency of multiple representations at once.

Summary of MVC

- Problem
  - Interactive Event-Driven Systems
- Solution (Pattern Composition)
  - Observer Pattern + Strategy Pattern + …
- Popular patterns in many real frameworks
  - Django, Rails, .NET MVC…

